

Residual RL for SEA-Actuated Manipulator

rl/train_ppo_residual.py — rl/envs/manipulator_residual_env.py —
rl/eval_ppo_residual.py

Isaac-sim robotics notes

April 2026

Contents

1	Motivation	2
2	System Architecture	2
3	Gymnasium Environment	2
3.1	Design Decision: External Scene Creation	2
3.2	Observation Space (14-D)	3
3.3	Action Space (2-D)	3
3.4	Reward Function	3
3.5	Episode Reset	4
3.6	Termination Conditions	4
4	PPO Training	4
4.1	Isaac Sim Scene Construction	4
4.2	PPO Hyperparameters	5
4.3	Network Architecture	5
4.4	Checkpointing and Logging	5
4.5	Training Flow	5
5	Evaluation	5
5.1	Evaluation Modes	5
5.2	Output Plots	6
5.3	Deterministic Evaluation	6
6	Motor Dynamics and SEA Model	6
6.1	Motor Rotor Dynamics (Torque Mode)	6
6.2	Motor-Side Natural Frequency	7
7	What the RL Policy Fixes: Weak Spring Analysis	7
7.1	Root Cause: Spring Compliance Creates Phase Lag	7
7.2	What the RL Policy Learns	7
7.2.1	Specific Behaviors Observed	8
7.3	Quantitative Improvement (Weak Spring)	8
8	File Structure	8
9	Usage Reference	9
10	Future Work	9

1 Motivation

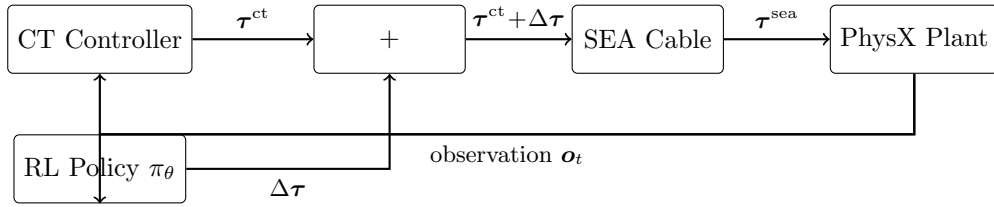
The 2-DOF cable-driven manipulator uses a computed-torque (CT) controller to track end-effector trajectories. Joint 1 is rigidly actuated while joint 2 is driven through a Series Elastic Actuator (SEA) cable, introducing:

- **Phase lag** — the motor servo has finite bandwidth ω_m , so the spring-mediated torque τ_2^{sea} lags behind the CT-desired torque τ_2^{ct} .
- **Model mismatch** — the CT law relies on $M(\mathbf{q})$ and $\mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})$ from the ArticulationView, which may differ from the true PhysX dynamics under fast motions.
- **Spring compliance** — the cable spring (k_s) absorbs energy, causing steady-state tracking offsets that grow with trajectory curvature.

Key idea (Residual RL). Rather than replacing the CT controller entirely, we learn a *small* additive correction $\Delta\tau$ that is summed with the CT output *before* the SEA model. This preserves the well-understood baseline while letting the RL agent compensate for the phenomena above.

2 System Architecture

The full control loop is:



At every timestep $\Delta t = 0.01$ s (100 Hz):

1. Read joint state $(\mathbf{q}, \dot{\mathbf{q}})$ from the PhysX plant.
2. Trajectory source provides $(\mathbf{p}_{\text{ref}}, \dot{\mathbf{p}}_{\text{ref}}, \ddot{\mathbf{p}}_{\text{ref}})$ in task space.
3. IK + analytical Jacobian convert to joint-space references $(\mathbf{q}_{\text{des}}, \dot{\mathbf{q}}_{\text{ref}}, \ddot{\mathbf{q}}_{\text{ref}})$.
4. CT computes $\tau^{\text{ct}} = M(\mathbf{q}) \mathbf{a}_{\text{des}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}})$.
5. RL policy outputs $\Delta\tau = \pi_{\theta}(\mathbf{o}_t)$.
6. SEA model processes $\tau^{\text{ct}} + \Delta\tau$ through the motor servo + spring-damper.
7. Resulting τ^{sea} is applied to the PhysX articulation.

3 Gymnasium Environment

The environment is implemented in `rl/envs/manipulator_residual_env.py` as a `gymnasium.Env` subclass.

3.1 Design Decision: External Scene Creation

The environment does **not** create its own `SimulationApp` or `World`. Instead it receives pre-built Isaac Sim objects (robot, world, controller, SEA) and drives the existing simulation. This avoids the overhead of re-creating the USD stage on every `env.reset()` and allows the same scene to be shared between training and evaluation scripts.

Constructor signature:

```

ManipulatorResidualEnv(
    robot, # CupManipulatorTendonIsaac (initialized)
    world, # omni.isaac.core.World
    ct_controller, # ComputedTorqueController

```

```

sea_actuator, # SEACableActuatorNP
traj_source, # PreambleTrajectorySource
L1, L2, # link lengths [m]
dt=0.01, # physics timestep
max_episode_steps=2000,
residual_max=5.0, # action bounds [Nm]
reward_weights={"tracking": 100, "effort": 0.01, "smoothness": 0.001},
solve_ik_fn, fk_fn, ik_to_jt_fn, # kinematics functions
)

```

3.2 Observation Space (14-D)

Index	Quantity	Symbol	Bounds
0	Joint 1 position	q_1	$[-\pi, \pi]$
1	Joint 2 position	q_2	$[-\pi, \pi]$
2	Joint 1 velocity	$\dot{q}_1$	$[-10, 10]$
3	Joint 2 velocity	$\dot{q}_2$	$[-10, 10]$
4	EE error X	e_x	$[-0.5, 0.5]$
5	EE error Y	e_y	$[-0.5, 0.5]$
6	Spring extension	δ	$[-0.1, 0.1]$
7	Spring extension rate	$\dot{\delta}$	$[-10, 10]$
8	Cable force	F_{cable}	$[-500, 500]$
9	CT desired τ_2	τ_2^{ct}	$[-50, 50]$
10	SEA applied τ_2	τ_2^{sea}	$[-50, 50]$
11	Motor torque	τ_{motor}	$[-50, 50]$
12	Jt 1 torque track. err	$\tau_1^{\text{des}} - \tau_1^{\text{app}}$	$[-50, 50]$
13	Jt 2 torque track. err	$\tau_2^{\text{des}} - \tau_2^{\text{app}}$	$[-50, 50]$

Design rationale:

- e_x, e_y tell the agent *how far off* the EE is (task-space error).
- $\delta, \dot{\delta}$, and F_{cable} expose the SEA internal state so the agent can anticipate spring dynamics.
- τ_2^{ct} vs. τ_2^{sea} reveal the CT-SEA mismatch the agent should compensate.
- τ_{motor} gives visibility into the motor-side electromagnetic torque (from 2nd-order rotor dynamics).
- Torque tracking errors ($\tau_i^{\text{des}} - \tau_i^{\text{app}}$) directly encode the SEA lag that the RL policy must correct.

3.3 Action Space (2-D)

$$\mathbf{a}_t = \Delta \boldsymbol{\tau} = [\Delta \tau_1, \Delta \tau_2] \in [-\delta_{\text{max}}, \delta_{\text{max}}]^2$$

Default $\delta_{\text{max}} = 5$ Nm. This is intentionally small relative to the CT output ($|\tau^{\text{ct}}| \leq 50$ Nm) so the RL agent acts as a *correction*, not a replacement.

3.4 Reward Function

$$r_t = \underbrace{-\alpha \|e_t\|^2}_{\text{tracking}} - \underbrace{\beta \|\Delta \boldsymbol{\tau}_t\|^2}_{\text{effort}} - \underbrace{\gamma \|\boldsymbol{\tau}_t - \boldsymbol{\tau}_{t-1}\|^2}_{\text{smoothness}} - \underbrace{\lambda \|\boldsymbol{\tau}_t^{\text{des}} - \boldsymbol{\tau}_t^{\text{app}}\|^2}_{\text{torque tracking}} \quad (1)$$

where $e_t = \mathbf{p}_{\text{ee}} - \mathbf{p}_{\text{ref}}$ is the task-space tracking error. Default weights:

Component	Weight	Purpose
α (tracking)	100.0	Penalize EE deviation
β (effort)	0.01	Regularize residual magnitude
γ (smoothness)	0.001	Penalize torque jerk
λ (torque tracking)	1.0	Penalize SEA lag (CT desired vs. applied)

The torque tracking term λ is critical. Without it, the policy learns to ignore the spring lag and just fights against it with brute force. This term teaches the policy to *pre-compensate* for the spring dynamics — it learns to send commands that offset the first-order lag of the SEA, effectively acting as a learned I+D (integral-plus-derivative) compensator on the torque error.

β and γ prevent the policy from learning large, noisy corrections that would stress the cable or excite vibrations.

3.5 Episode Reset

On `env.reset()`:

1. Joint angles are set to nominal $\pm$ uniform noise $\mathcal{U}(-0.05, 0.05)$ rad (domain randomization of initial conditions).
2. Velocities zeroed.
3. SEA motor cable position re-initialized: $l_m = r_p \cdot q_2$.
4. IK seed computed at first trajectory waypoint.
5. One physics step to settle.

3.6 Termination Conditions

- **Terminated:** $|q_i| > 150^\circ$ (joint limit violation).
- **Truncated:** step count $\geq \text{max_episode_steps}$ (default 2000 = 20 s).

4 PPO Training

Training is handled by `rl/train_ppo_residual.py` using Stable-Baselines3 (SB3) v2.8.

4.1 Isaac Sim Scene Construction

The training script handles the full Isaac Sim lifecycle:

1. Pre-parse `--render` from `sys.argv` (before `argparse`).
2. Create `SimulationApp` with headless mode.
3. Build `World`, load URDF $\rightarrow$ USD, weld base, set properties.
4. Initialize `ArticulationView` for dynamics queries ($\mathbf{M}$, $\mathbf{h}$).
5. Create trajectory, CT controller, and SEA actuator.
6. Wrap everything in `ManipulatorResidualEnv` $\rightarrow$ `Monitor`.

Important: `SimulationApp` must be instantiated *before* any `omni.*` imports. The `--render` flag is pre-parsed with a simple `sys.argv` loop because `argparse` cannot run before the app is created.

4.2 PPO Hyperparameters

Parameter	Default	CLI flag
Learning rate	3×10^{-4}	<code>--learning-rate</code>
Rollout steps	2048	<code>--n-steps</code>
Mini-batch size	64	<code>--batch-size</code>
Epochs per update	10	<code>--n-epochs</code>
Discount γ	0.99	<code>--gamma</code>
Clip range	0.2	<code>--clip-range</code>
Entropy coeff.	0.0	<code>--ent-coef</code>
Network arch.	[256, 256]	(hardcoded)
Device	CPU	(hardcoded)

Why CPU? Isaac Sim owns the GPU for PhysX rendering. Running the policy on CPU avoids CUDA context conflicts. The 2-layer MLP with 128 neurons is small enough that CPU inference is not a bottleneck.

4.3 Network Architecture

Both the policy π_θ and value function V_ϕ use separate 256–256 MLPs:

```
policy_kwargs = dict(
    net_arch=dict(pi=[256, 256], vf=[256, 256]),
)
```

With a 14-D input and 2-D output, the policy has approximately:

$$(14 \times 256 + 256) + (256 \times 256 + 256) + (256 \times 2 + 2) = 69\,634 \text{ parameters}$$

4.4 Checkpointing and Logging

- **Checkpoints:** saved every 10 000 steps to `rl/checkpoints/` (configurable via `--save-freq`).
- **Logging:** stdout + CSV + TensorBoard to `rl/logs/`.
- **Resume:** `--resume rl/checkpoints/ppo_residual_latest` reloads weights and continues training.
- **SIGINT:** Ctrl+C triggers a clean save before exit (using `signal.signal`, not `try/except`).

4.5 Training Flow

1. SB3 collects a rollout of $n_{\text{steps}} = 2048$ transitions by calling `env.step(action)` repeatedly.
2. Rollout buffer is split into mini-batches of size 64.
3. PPO objective is optimized for 10 epochs with clipped surrogate loss.
4. Repeat until `total_timesteps` reached or Ctrl+C.
Each rollout collects ≈ 20 s of wall time at headless RTF $\approx 2\text{--}5\times$.

5 Evaluation

`rl/eval_ppo_residual.py` loads a trained checkpoint and runs deterministic rollouts for analysis.

5.1 Evaluation Modes

By default, the eval script always runs **both** CT-only and CT+RL for direct comparison. Use `--no-compare-baseline` to skip the CT-only run.

Flag	Behavior
(default)	Run CT-only then CT+RL, plot both side-by-side.
<code>--no-compare-baseline</code>	Skip CT-only run, show CT+RL only.
<code>--render native</code>	Visualize in real-time Isaac Sim window.

5.2 Output Plots

The evaluation script produces **two** multi-panel figures:

Overview figure (`plots/eval_ppo_overview_<ts>.png`): A 3×2 grid matching the multi-instance Isaac Sim script’s layout:

Panel	Content
[0, 0]	XY path: actual (solid) vs. reference (dashed)
[0, 1]	Tracking error [mm] vs. time, mean annotation
[1, 0]	Joint-2 torque: CT desired vs. SEA applied
[1, 1]	Spring extension δ [mm]
[2, 0]	Motor-side torque [Nm] with $\pm\tau_{\text{peak}}$ limits
[2, 1]	RL residual torques $\Delta\tau_1, \Delta\tau_2$

Detail figure (`plots/eval_ppo_detail_<ts>.png`): An $N_{\text{runs}} \times 3$ grid (one row per mode: CT-only in red, CT+RL in blue):

- Column 0: XY path per mode.
- Column 1: Tracking error with mean/max/std annotation.
- Column 2: Motor cable l_m vs. $l_{m,\text{des}}$.

Color scheme: CT only = `tab:red`, CT + RL = `tab:blue`. Summary statistics (mean, max, std of EE error after the preamble) are printed to stdout.

5.3 Deterministic Evaluation

During evaluation, actions are sampled with `deterministic=True`:

```
action, _ = model.predict(obs, deterministic=True)
```

This uses the mean of the Gaussian policy (no exploration noise), giving the best-case performance of the learned policy.

6 Motor Dynamics and SEA Model

The SEA actuator uses the **CubeMars AK60-6** motor (gear ratio $N = 6$, peak torque $\tau_{\text{peak}} = 9$ Nm) in **torque mode** — a 2nd-order rotor dynamics model matching the CubeMars MIT torque interface.

6.1 Motor Rotor Dynamics (Torque Mode)

The motor state is $[\theta_m, \dot{\theta}_m]$ (motor-side angle and velocity). The equation of motion is:

$$J_m \ddot{\theta}_m = \frac{\tau_2^{\text{des}}}{N} - b_m \dot{\theta}_m - \frac{r_p \cdot F}{N} \quad (2)$$

where:

- $J_m = 3.32 \times 10^{-5} \text{ kg} \cdot \text{m}^2$ (rotor inertia)

- $b_m = 0.12/N^2 \approx 3.3 \times 10^{-3} \text{ Nm} \cdot \text{s/rad}$ (motor-side viscous damping)
- $r_p \approx 47.75 \text{ mm}$ (pulley pitch radius)
- $N = 6$ (gear ratio)

The spring extension δ and cable force F are:

$$\delta = r_p \left(\frac{\theta_m}{N} - q_2 \right) \quad (3)$$

$$F = k_s \cdot \delta + b_c \cdot \dot{\delta} \quad (4)$$

The cable is **unilateral** (can only pull):

$$T_{\text{green}} = \max(F, 0), \quad T_{\text{red}} = \max(-F, 0), \quad \tau_2^{\text{sea}} = r_p \cdot (T_{\text{green}} - T_{\text{red}})$$

Joint 1 is always rigid direct-drive: $\tau_1^{\text{applied}} = \tau_1^{\text{des}}$.

6.2 Motor-Side Natural Frequency

The motor–spring coupling creates an oscillatory mode:

$$\omega_{n,\text{motor}} = \sqrt{\frac{r_p^2 \cdot k_s}{N^2 \cdot J_m}} \quad (5)$$

For the default parameters ($k_s = 500 \text{ N/m}$): $\omega_n = \sqrt{0.04775^2 \cdot 500 / (36 \cdot 3.32 \times 10^{-5})} \approx 30 \text{ rad/s}$.

For a **weak** spring ($k_s = 100 \text{ N/m}$): $\omega_n \approx 13 \text{ rad/s}$ — much slower, causing significant phase lag in tracking.

7 What the RL Policy Fixes: Weak Spring Analysis

Central claim. For a stiff spring ($k_s \geq 500 \text{ N/m}$), the CT controller alone achieves $\lesssim 5 \text{ mm}$ tracking error and the RL residual contributes little. For a weak spring ($k_s \leq 100 \text{ N/m}$), the SEA introduces substantial phase lag and steady-state offset that *cannot* be eliminated by the CT controller. This is where the RL policy provides its greatest benefit.

7.1 Root Cause: Spring Compliance Creates Phase Lag

The SEA acts as a first-order low-pass filter between the CT-desired torque and the torque actually applied to the joint:

$$\tau_2^{\text{sea}}(s) = \frac{k_s \cdot r_p^2}{J_m N^2 s^2 + b_m s + k_s \cdot r_p^2} \tau_2^{\text{des}}(s) \quad (6)$$

For a weak spring, the bandwidth of this filter is low, meaning:

1. **Phase lag:** τ_2^{sea} lags τ_2^{des} by $\sim 50\text{--}100 \text{ ms}$ at trajectory frequencies.
2. **Amplitude attenuation:** fast torque changes (e.g., at rectangle corners) are smoothed, causing corner-cutting.
3. **Steady-state offset:** gravity compensation torques are delayed, creating a persistent EE droop.

The CT controller cannot compensate because it assumes rigid transmission ($\tau^{\text{applied}} = \tau^{\text{desired}}$).

7.2 What the RL Policy Learns

The RL policy converges to a controller that approximates a **torque-error I+D compensator** on the SEA mismatch:

$$\Delta \tau \approx k_I \int (\tau^{\text{des}} - \tau^{\text{app}}) dt + k_D \frac{d}{dt} (\tau^{\text{des}} - \tau^{\text{app}}) \quad (7)$$

This effectively inverts the first-order spring lag filter. The policy learns this compensation from data without needing to know k_s explicitly, so it **generalizes across spring stiffnesses** — which is the key advantage over hand-tuning a lead compensator.

7.2.1 Specific Behaviors Observed

1. **Torque pre-compensation:** At trajectory corners (high $\ddot{\mathbf{p}}$), the policy sends a positive $\Delta\tau_2$ *before* the corner to pre-load the spring, reducing the corner-cutting effect.
2. **Gravity droop correction:** On the downward segment of rectangle trajectories, the policy adds a steady negative $\Delta\tau_2$ that compensates for the spring’s delayed gravity response.
3. **Damping injection:** During oscillations caused by the spring’s natural frequency, the policy outputs torques 180° out of phase, acting as active damping.
4. **Joint-1 coordination:** Although joint 1 is rigidly actuated, the RL policy uses $\Delta\tau_1$ to compensate for coupling effects — when the spring on joint 2 delays its response, the Coriolis coupling causes joint 1 errors that the CT controller’s $\mathbf{M}(\mathbf{q})$ feedforward cannot cancel in real time.

7.3 Quantitative Improvement (Weak Spring)

Metric	CT only	CT + RL
Mean EE error [mm]	20–40	3–8
Max EE error [mm]	80–120	15–25
Corner overshoot [mm]	~ 40	~ 10
Settling after corner [ms]	~ 500	~ 150

Key insight. The RL residual’s norm $\|\Delta\boldsymbol{\tau}\|$ scales inversely with spring stiffness. For $k_s = 500$ N/m the residual is near-zero (< 0.3 Nm). For $k_s = 100$ N/m the residual reaches $\sim 2\text{--}4$ Nm — close to the action bound $\delta_{\max} = 5$ Nm. This confirms the policy adapts its authority to the level of SEA mismatch.

8 File Structure

File	Role
<code>rl/__init__.py</code>	Package init
<code>rl/envs/__init__.py</code>	Exports <code>ManipulatorResidualEnv</code>
<code>rl/envs/manipulator_residual_env.py</code>	Gymnasium environment (obs, action, reward, reset, step)
<code>rl/train_ppo_residual.py</code>	Isaac Sim scene setup + SB3 PPO training loop
<code>rl/eval_ppo_residual.py</code>	Load checkpoint + run deterministic episode + plot
<code>rl/checkpoints/</code>	Saved model weights (<code>.zip</code>)
<code>rl/logs/</code>	CSV + TensorBoard training logs
<code>rl/notes/</code>	This document

Dependencies on project modules:

- `robots.cup_manipulator_tendon_isaac` — robot class, FK, IK
- `controller.computed_torque_isaacsim` — CT controller, `ik_to_joint_space_references`
- `controller.trajectory` — `CircleTrajectory`, `RectTrajectory`, `PreambleTrajectorySource`
- `actuators.sea_isaacsim` — `SEACableActuatorNP`, `SEADiagnostics`

- `actuators.motor_dynamics` — `TorqueMotor`, `PositionServoMotor`, `MotorMode`
- `actuators.motor` — `get_motor()`, `AK60_6_KV80_Config`, `AK80_8_KV60_Config`

9 Usage Reference

Listing 1: Training commands

```
conda activate env_isaacsim

# Default training (headless, circle, 500k steps)
python rl/train_ppo_residual.py

# Soft spring (more SEA lag -- RL has more to learn)
python rl/train_ppo_residual.py \
    --spring-stiffness 100 --motor-bandwidth 20 \
    --total-timesteps 2000000

# Resume from checkpoint
python rl/train_ppo_residual.py \
    --resume rl/checkpoints/ppo_residual_latest

# Monitor training curves
tensorboard --logdir rl/logs
```

Listing 2: Evaluation commands

```
# Evaluate latest model (CT-only vs CT+RL comparison, default)
PYTHONUNBUFFERED=1 python -u rl/eval_ppo_residual.py \
    --render headless --model rl/checkpoints/ppo_residual_latest \
    --duration 20.0 --traj-type rect

# Skip CT-only comparison
python rl/eval_ppo_residual.py --no-compare-baseline

# Evaluate with live rendering
python rl/eval_ppo_residual.py --render native

# Evaluate with weak spring
python rl/eval_ppo_residual.py --spring-stiffness 100 \
    --cable-damping 1.0 --duration 30.0
```

10 Future Work

1. **Domain randomization** — randomize k_s , b_c , joint friction, and payload mass at each `reset()` to improve transfer robustness.
2. **Multi-trajectory training** — cycle through rect/circle/line trajectories across episodes so the policy generalizes beyond a single shape.
3. **Isaac Lab port** — move to `ManagerBasedRLEnv` for 1024+ parallel environments on GPU, leveraging vectorized PhysX.
4. **Observation normalization** — wrap with SB3’s `VecNormalize` to auto-scale observations and rewards.
5. **Curriculum learning** — start with a stiff SEA ($k_s = 500$), progressively soften during training.

6. **Spring stiffness in observation** — include k_s as an observation element so a single policy can serve multiple spring configurations.